Reinforcement Learning with Amazon SageMaker RL

Reinforcement learning (RL) is a machine learning technique that attempts to learn a strategy, called a policy, that optimizes an objective for an agent acting in an environment. For example, the agent might be a robot, the environment might be a maze, and the goal might be to successfully navigate the maze in the smallest amount of time. In RL, the agent takes an action, observes the state of the environment, and gets a reward based on the value of the current state of the environment. The goal is to maximize the long-term reward that the agent receives as a result of its actions. RL is well-suited for solving problems where an agent can make autonomous decisions.

Topics

- Why is Reinforcement Learning Important? (p. 454)
- Markov Decision Process (MDP) (p. 454)
- Key Features of Amazon SageMaker RL (p. 455)
- Sample RL Workflow Using Amazon SageMaker RL (p. 457)
- RL Environments in Amazon SageMaker (p. 458)
- Distributed Training with Amazon SageMaker RL (p. 459)
- Hyperparameter Tuning with Amazon SageMaker RL (p. 460)

Why is Reinforcement Learning Important?

RL is well-suited for solving large, complex problems. For example, supply chain management, HVAC systems, industrial robotics, game artificial intelligence, dialog systems, and autonomous vehicles. Because RL models learn by a continuous process of receiving rewards and punishments for every action taken by the agent, it is possible to train systems to make decisions under uncertainty and in dynamic environments.

Markov Decision Process (MDP)

RL is based on models called Markov Decision Processes (MDPs). An MDP consists of a series of time steps. Each time step consists of the following:

Environment

Defines the space in which the RL model operates. This can be either a real-world environment or a simulator. For example, if you train a physical autonomous vehicle on a physical road, that would be a real-world environment. If you train a computer program that models an autonomous vehicle driving on a road, that would be a simulator.

State

Specifies all information about the environment and past steps that is relevant to the future. For example, in an RL model in which a robot can move in any direction at any time step, then the

position of the robot at the current time step is the state, because if we know where the robot is, it isn't necessary to know the steps it took to get there.

Action

What the agent does. For example, the robot takes a step forward.

Reward

A number that represents the value of the state that resulted from the last action that the agent took. For example, if the goal is for a robot to find treasure, the reward for finding treasure might be 5, and the reward for not finding treasure might be 0. The RL model attempts to find a strategy that optimizes the cumulative reward over the long term. This strategy is called a *policy*.

Observation

Information about the state of the environment that is available to the agent at each step. This might be the entire state, or it might be just a part of the state. For example, the agent in a chess-playing model would be able to observe the entire state of the board at any step, but a robot in a maze might only be able to observe a small portion of the maze that it currently occupies.

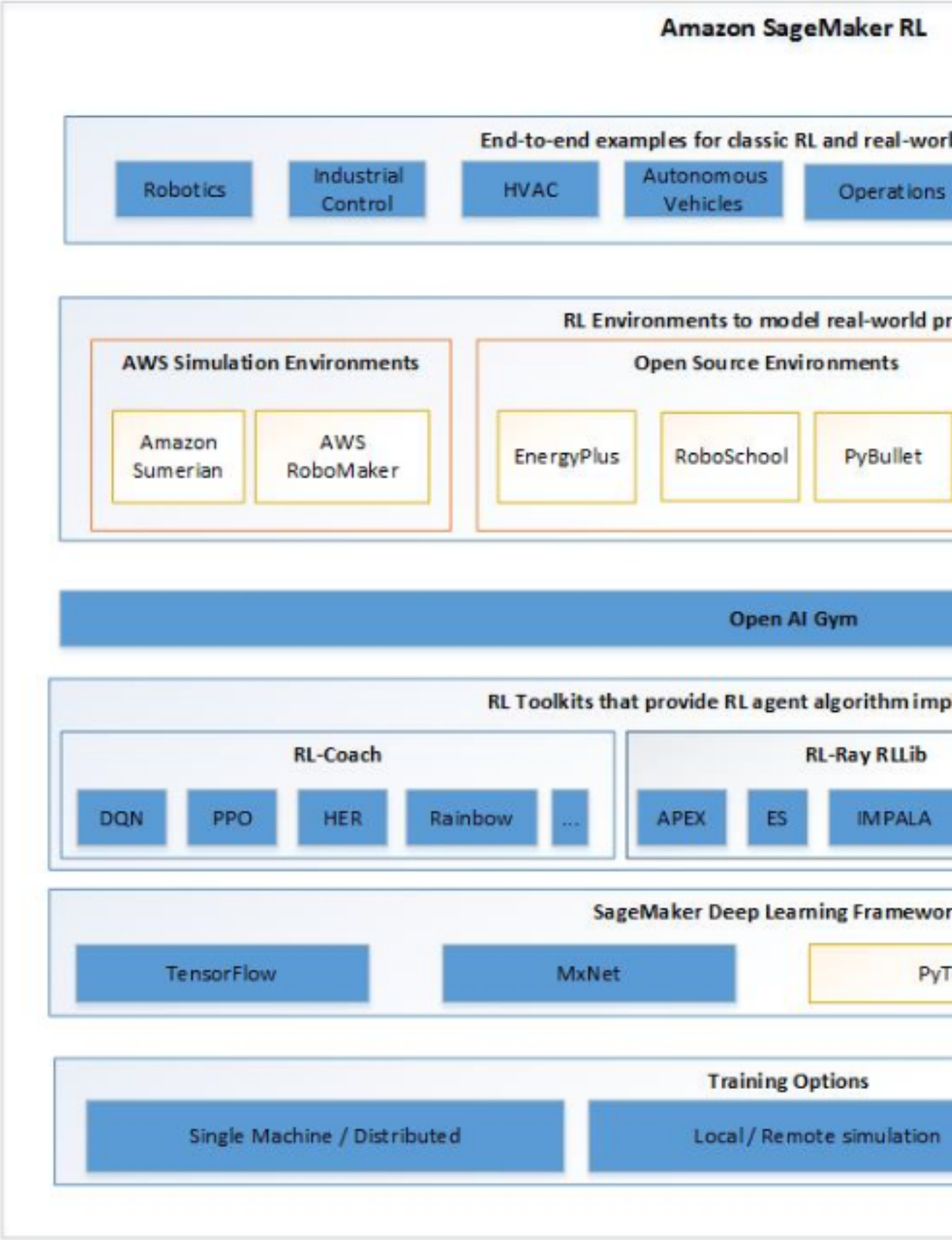
Typically, training in RL consists of many *episodes*. An episode consists of all of the time steps in an MDP from the initial state until the environment reaches the terminal state.

Key Features of Amazon SageMaker RL

To train RL models in Amazon SageMaker RL, use the following components:

- A deep learning (DL) framework. Currently, Amazon SageMaker supports RL in TensorFlow and Apache MXNet.
- An RL toolkit. An RL toolkit manages the interaction between the agent and the environment, and provides a wide selection of state of the art RL algorithms. Amazon SageMaker supports the Intel Coach and Ray RLlib toolkits. For information about Intel Coach, see https://nervanasystems.github.io/coach/. For information about Ray RLlib, see https://ray.readthedocs.io/en/latest/rllib.html.
- An RL environment. You can use custom environments, open-source environments, or commercial environments. For information, see RL Environments in Amazon SageMaker (p. 458).

The following diagram shows the RL components that are supported in Amazon SageMaker RL.



SageMaker supported

Customer BYO

Sample RL Workflow Using Amazon SageMaker RL

The following example describes the steps for developing RL models using Amazon SageMaker RL.

For complete code examples, see the sample notebooks at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement-learning.

1. **Formulate the RL problem**—First, formulate the business problem into an RL problem. For example, auto scaling enables services to dynamically increase or decrease capacity depending on conditions that you define. Currently, this requires setting up alarms, scaling policies, and thresholds, and other manual steps. To solve this with RL, we define the components of the Markov Decision Process:

   a. **Objective**—Scale instance capacity so that it matches the desired load profile.

   b. **Environment**—A custom environment that includes the load profile. It generates a simulated load with daily and weekly variations and occasional spikes. The simulated system has a delay between when new resources are requested and when they become available for serving requests.

   c. **State**—The current load, number of failed jobs, and number of active machines

   d. **Action**—Remove, add, or keep the same number of instances.

   e. **Reward**—A positive reward for successful transactions, a high penalty for failing transactions beyond a specified threshold.

2. **Define the RL environment**—The RL environment can be the real world where the RL agent interacts or a simulation of the real world. You can connect open source and custom environments developed using Gym interfaces, and commercial simulation environments such as MATLAB and Simulink.

3. **Define the presets**—The presets configure the RL training jobs and define the hyperparameters for the RL algorithms.

4. **Write the training code**—Write training code as a Python script and pass the script to an Amazon SageMaker training job. In your training code, import the environment files and the preset files, and then define the `main()` function.

5. **Train the RL Model**— Use the Amazon SageMaker `RLEstimator` in the Amazon SageMaker Python SDK to start an RL training job. If you are using local mode, the training job runs on the notebook instance. When you use Amazon SageMaker for training, you can select GPU or CPU instances. Store the output from the training job in a local directory if you train in local mode, or on Amazon S3 if you use Amazon SageMaker training.

   For information about using the Amazon SageMaker Python SDK for RL, see https://github.com/aws/sagemaker-python-sdk/blob/master/src/sagemaker/rl/README.rst .

   The `RLEstimator` requires the following information as parameters.

   a. The source directory where the environment, presets, and training code are uploaded.

   b. The path to the training script.

   c. The RL toolkit and deep learning framework you want to use. This automatically resolves to the Amazon ECR path for the RL container.

   d. The training parameters, such as the instance count, job name, and S3 path for output.

   e. Metric definitions that you want to capture in your logs. These can also be visualized in CloudWatch and in Amazon SageMaker notebooks.

6. **Visualize training metrics and output**—After a training job that uses an RL model completes, you can view the metrics you defined in the training jobs in CloudWatch,. You can also plot the metrics in a notebook by using the Amazon SageMaker Python SDK analytics library. Visualizing metrics helps you understand how the performance of the model as measured by the reward improves over time.

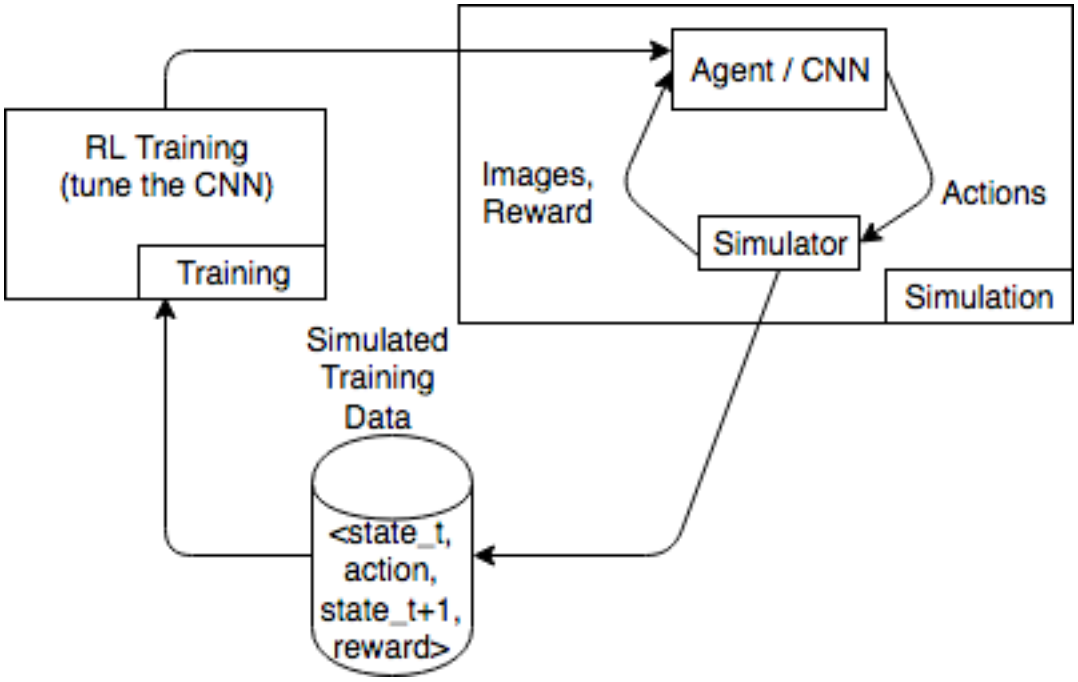
Note
If you train in local mode, you can't visualize metrics in CloudWatch.

7. **Evaluate the model**—Checkpointed data from the previously trained models can be passed on for evaluation and inference in the checkpoint channel. In local mode, use the local directory. In Amazon SageMaker training mode, you need to upload the data to S3 first.
8. **Deploy RL models**—Finally, deploy the trained model on an endpoint hosted on Amazon SageMaker or on an Edge device by using AWS IoT Greengrass.

RL Environments in Amazon SageMaker

Amazon SageMaker RL uses environments to mimic real-world scenarios. Given the current state of the environment and an action taken by the agent or agents, the simulator processes the impact of the action, and returns the next state and a reward. Simulators are useful in cases where it is not safe to train an agent in the real world (for example, flying a drone) or if the RL algorithm takes a long time to converge (for example, when playing chess).

The following diagram shows an example of the interactions with a simulator for a car racing game.



The simulation environment consists of an agent and a simulator. Here, a convolutional neural network (CNN) consumes images from the simulator and generates actions to control the game controller. With multiple simulations, this environment generates training data of the form `state_t`, `action`, `state_t+1`, and `reward_t+1`. Defining the reward is not trivial and impacts the RL model quality. We want to provide a few examples of reward functions, but would like to make it user-configurable.

Topics

- Use OpenAI Gym Interface for Environments in Amazon SageMaker RL (p. 459)
- Use Open Source Environments (p. 459)
- Use Commercial Environments (p. 459)

Use OpenAI Gym Interface for Environments in Amazon SageMaker RL

To use OpenAI Gym environments in Amazon SageMaker RL, use the following API elements. For more information about OpenAI Gym, see https://gym.openai.com/docs/.

- `env.action_space`—Defines the actions the agent can take, specifies whether each action is continuous or discrete, and specifies the minimum and maximum if the action is continuous.
- `env.observation_space`—Defines the observations the agent receives from the environment, as well as minimum and maximum for continuous observations.
- `env.reset()`—Initializes a training episode. The `reset()` function returns the initial state of the environment, and the agent uses the initial state to take its first action. The action is then sent to the `step()` repeatedly until the episode reaches a terminal state. When `step()` returns `done = True`, the episode ends. The RL toolkit re-initializes the environment by calling `reset()`.
- `step()`—Takes the agent action as input and outputs the next state of the environment, the reward, whether the episode has terminated, and an `info` dictionary to communicate debugging information. It is the responsibility of the environment to validate the inputs.
- `env.render()`Used for environments that have visualization. The RL toolkit calls this function to capture visualizations of the environment after each call to the `step()` function.

Use Open Source Environments

You can use open source environments, such as EnergyPlus and RoboSchool, in Amazon SageMaker RL by building your own container. For more information about EnergyPlus, see https://energyplus.net/. For more information about RoboSchool, see https://github.com/openai/roboschool. The HVAC and RoboSchool examples in the samples repository at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement_learning show how to build a custom container to use with Amazon SageMaker RL:

Use Commercial Environments

You can use commercial environments, such as MATLAB and Simulink, in Amazon SageMaker RL by building your own container. You need to manage your own licenses.

Distributed Training with Amazon SageMaker RL

Amazon SageMaker RL supports multi-core and multi-instance distributed training. Depending on your use case, training and/or environment rollout can be distributed. For example, Amazon SageMaker RL works for the following distributed scenarios:

- Single training instance and multiple rollout instances of the same instance type. For an example, see the Neural Network Compression example in the Amazon SageMaker examples repository at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement_learning.
- Single trainer instance and multiple rollout instances, where different instance types for training and rollouts. For an example, see the AWS DeepRacer / AWS RoboMaker example in the Amazon SageMaker examples repository at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement_learning.
- Single trainer instance that uses multiple cores for rollout. For an example, see the Roboschool example in the Amazon SageMaker examples repository at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement_learning. This is useful if the simulation environment is light-weight and can run on a single thread.

- Multiple instances for training and rollouts. For an example, see the Roboschool example in the Amazon SageMaker examples repository at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/reinforcement_learning.

Hyperparameter Tuning with Amazon SageMaker RL

You can run a hyperparameter tuning job to optimize hyperparameters for Amazon SageMaker RL. The Roboschool example in the sample notebooks at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/'reinforcement-learning shows how you can do this with RL Coach. The launcher script shows how you can abstract parameters from the Coach preset file and optimize them.